

WHAT IF?

COUNTERFACTUAL INTELLIGENCE ENGINE

GenAI-Powered Scenario Simulation & Decision Support Workflow

STUDENT	VADDI THEJESWAR REDDY
REGISTER NO.	99230040793
ASSESSMENT	Mini Assessment — Prompt Workflow Design
PROJECT	What If? Counterfactual Intelligence Engine

Assignment basis: the project document provides the What If? workflow, while the accompanying workflow document requires a structured sequence of setup/planning, drafting/execution, and a mandatory critique-and-refinement gate.

01 | Concept

The **What If? Counterfactual Intelligence Engine** converts natural-language hypothetical questions into structured scenarios, identifies variables and assumptions, retrieves supporting context, simulates multiple plausible outcomes, and explains the differences between possible futures.

The central problem is that people often make decisions without comparing the consequences of changing one important variable. Instead of returning one unsupported prediction, this workflow uses GenAI, structured models, evidence, and simulation to investigate counterfactual situations before action is taken.

02 | Objective

Design a sequential prompt workflow that interprets a user's counterfactual question, extracts variables and constraints, constructs causal context, retrieves relevant evidence, generates plausible scenarios, evaluates uncertainty, and presents an understandable comparison of outcomes.

03 | Workflow Overview

1. Interpret	2. Structure	3. Model	4. Simulate	5. Explain
Scenario intent	Scenario specification	Causal context	Scenario set	Decision insight

04 | Workflow Steps

- 1. Scenario Interpretation:** Understand the hypothetical, baseline situation, decision, and intended scope. Output: Scenario Intent.
- 2. Variable & Constraint Extraction:** Convert the scenario into variables, values, constraints, assumptions, affected entities, and time horizon. Output: Scenario Specification.
- 3. Causal Context & Evidence:** Connect relevant variables and retrieve supporting knowledge or data. Separate evidence from assumptions. Output: Causal Context.
- 4. Counterfactual Simulation:** Explore best-case, likely-case, and worst-case paths under changed assumptions. Output: Scenario Set.
- 5. Uncertainty & Explanation:** Compare outcomes, identify sensitive variables, and explain assumptions and limitations. Output: Decision Insight.

05 | Prompt Workflow

Following the assignment's required progression, the prompts below are dependent: each step uses the output of the previous step rather than independently recreating the task.

Step 1 — Structuring & Planning (The Setup)

Goal: Establish the system role, interpret the counterfactual, and organize the scenario.

Prompt 1: Act as a counterfactual scenario analyst. Read the user's hypothetical question. Identify the baseline situation, the variable being changed, the main decision, affected entities, scope, and time horizon. Extract the intended scenario without predicting an outcome yet. Return a structured **Scenario Intent** containing: baseline, changed variable, decision, scope, affected entities, and time horizon.

Expected AI Output: A structured scenario intent that clearly states what is changing and what is being compared.

Step 2 — Drafting / Execution

Goal: Generate the structured scenario and plausible futures using only the outputs established in Step 1.

Prompt 2: Excellent. Now, using only the Scenario Intent from Step 1, build the counterfactual scenario. Extract variables, values, constraints, assumptions, affected entities, and time horizon. Represent the scenario as machine-readable data. Then build causal context by identifying relationships among variables and separating evidence from assumptions. Finally, generate best-case, likely-case, and worst-case plausible outcomes. Do not present any scenario as guaranteed.

Expected AI Output: A Structured Scenario, Causal Context, and Scenario Set containing multiple plausible futures rather than one guaranteed prediction.

Step 3 — Critique and Refinement (The Required Gate)

Goal: Force the workflow to evaluate its own result against explicit quality criteria and correct any failures before finalization.

Prompt 3: Before finalizing, act as a strict senior reviewer. Review the Scenario Set and decision explanation against this checklist: (1) Were the baseline and changed variable clearly identified? (2) Are assumptions separated from evidence? (3) Are best-case, likely-case, and worst-case outcomes included? (4) Is uncertainty communicated instead of claiming certainty? (5) Are high-impact variables identified? (6) Does the final comparison explain why the outcomes differ? First list any failures or missing elements. Second, output the corrected **Decision Insight / Report** that fixes every issue.

Expected AI Output: A concise self-critique followed immediately by a refined decision-support report that satisfies all checklist requirements.

This three-stage structure demonstrates the required progression, dependency between steps, and mandatory evaluation gate described in the assignment document.

06 | Prompt Chaining Architecture

Step 1 establishes the hypothetical; Step 2 converts it into structured variables; Step 3 connects variables through causal context and evidence; Step 4 explores possible futures; and Step 5 turns the simulation into an understandable decision report.

STAGE	INPUT / CONTEXT	OUTPUT
1	Natural-language What-If question	Scenario Intent
2	Intent + baseline + constraints	Structured Scenario
3	Variables + evidence + relationships	Causal Context
4	Scenario + simulation assumptions	Multiple Future Outcomes
5	Outcomes + uncertainty + sensitivity	Decision Insight / Report

07 | Demonstration Input

Example Scenario: A small food business sells 100 meals per day at ■150 per meal. The owner asks: *“What if I increase the price by 20% for the next three months?”* The system treats the current situation as the baseline and identifies price, demand, revenue, customer retention, and competitor response as relevant variables.

08 | Expected Final Output & Workflow Result

The system produces a counterfactual comparison rather than claiming a guaranteed result. It can show a range of possible demand changes under different assumptions, calculate corresponding revenue ranges, identify assumptions responsible for the largest changes, and explain which additional data would improve confidence.

09 | Counterfactual Comparison

SCENARIO	ASSUMPTION	POSSIBLE EFFECT
Baseline	Price remains ■150	Reference demand and revenue
Optimistic	Demand decreases slightly	Higher revenue with limited churn
Likely	Demand decreases moderately	Revenue depends on demand elasticity
Pessimistic	Demand decreases sharply	Revenue may fall despite higher price

10 | Sample Support Execution Plan

PHASE	ACTION FOCUS	OUTPUT / METRIC
Phase 1	Scenario Understanding	Hypothetical and baseline extracted
Phase 2	Variable Modeling	Key variables and assumptions identified
Phase 3	Simulation	Multiple plausible futures generated
Phase 4	Sensitivity Analysis	High-impact variables ranked
Phase 5	Resolution	Comparison report and decision insights

11 | Key Benefits

- ✓ Converts vague hypothetical questions into structured, testable scenarios.
- ✓ Explores multiple plausible futures instead of producing one unsupported prediction.
- ✓ Separates evidence, assumptions, and simulation results for transparency.
- ✓ Identifies variables with the greatest influence on the outcome.
- ✓ Provides an interactive decision-support layer for planning and operations.

12 | Discussion Questions & Answers

1. What is the advantage of using dynamic scenario variables instead of hard-coding a case?

Dynamic variables allow the same workflow to process many hypothetical situations. Values such as {baseline}, {changed_variable}, {time_horizon}, and {constraints} can change without rewriting the workflow.

2. When would .format() be enough?

Python's .format() is sufficient for simple linear prompt construction where fixed instructions only need values such as a variable name, baseline value, or scenario description inserted.

3. When might Jinja2 be useful?

Jinja2 is useful when prompts need conditional or repeated structures, such as different instructions for financial variables, multiple time periods, or several alternative futures.

4. Why should prompts be versioned?

Prompt versions provide an audit trail for changes in interpretation and reasoning behavior. If a new prompt causes the system to omit assumptions or misread a scenario, earlier versions can be compared or restored.

5. What is a prompt regression in this system?

A prompt regression occurs when a prompt modification improves one scenario type but unintentionally reduces performance on previously reliable scenarios.

6. Why use the same evaluation set when comparing prompt versions?

A fixed evaluation set makes comparisons consistent. If both prompts and test scenarios change, it becomes difficult to determine whether performance changes came from the prompt or from different test cases.

13 | Technical Stack

LAYER	TECHNOLOGY
GenAI	LLM API / Local LLM
Backend	Python, FastAPI
Simulation	NumPy, Pandas
Knowledge	RAG / Vector Database
Frontend	React or Streamlit
Storage	PostgreSQL

14 | Final Outcome

The What If? Counterfactual Intelligence Engine translates natural-language hypotheticals into structured scenarios, explores plausible alternatives, and communicates the assumptions and uncertainty behind each outcome. It is designed to support reasoning about how changing a variable could change the future—not to claim certainty about what will happen.

Assignment Completion Note: The workflow above follows the supplied document's required pattern: setup/structuring → execution/drafting → strict critique and refinement. The final gate explicitly checks the workflow against measurable criteria before producing the corrected final report.